Aprendizagem em Conjunto (*Ensemble Learning*)

Gabriel P. Oliveira

Inteligência Artificial
2026/1

O que aprendemos até agora?

- Aprendizado preditivo (classificação/regressão)
- Árvores de Decisão
  - Treinamento a partir de conjunto de dados
  - Principais algoritmos
- Métricas de avaliação (acurácia, precisão, *recall*, etc.)
- Métodos de amostragem para avaliação
  - Holdout
  - Amostragem aleatória
  - Validação cruzada
  - *Bootstraping*

O que aprendemos até agora?

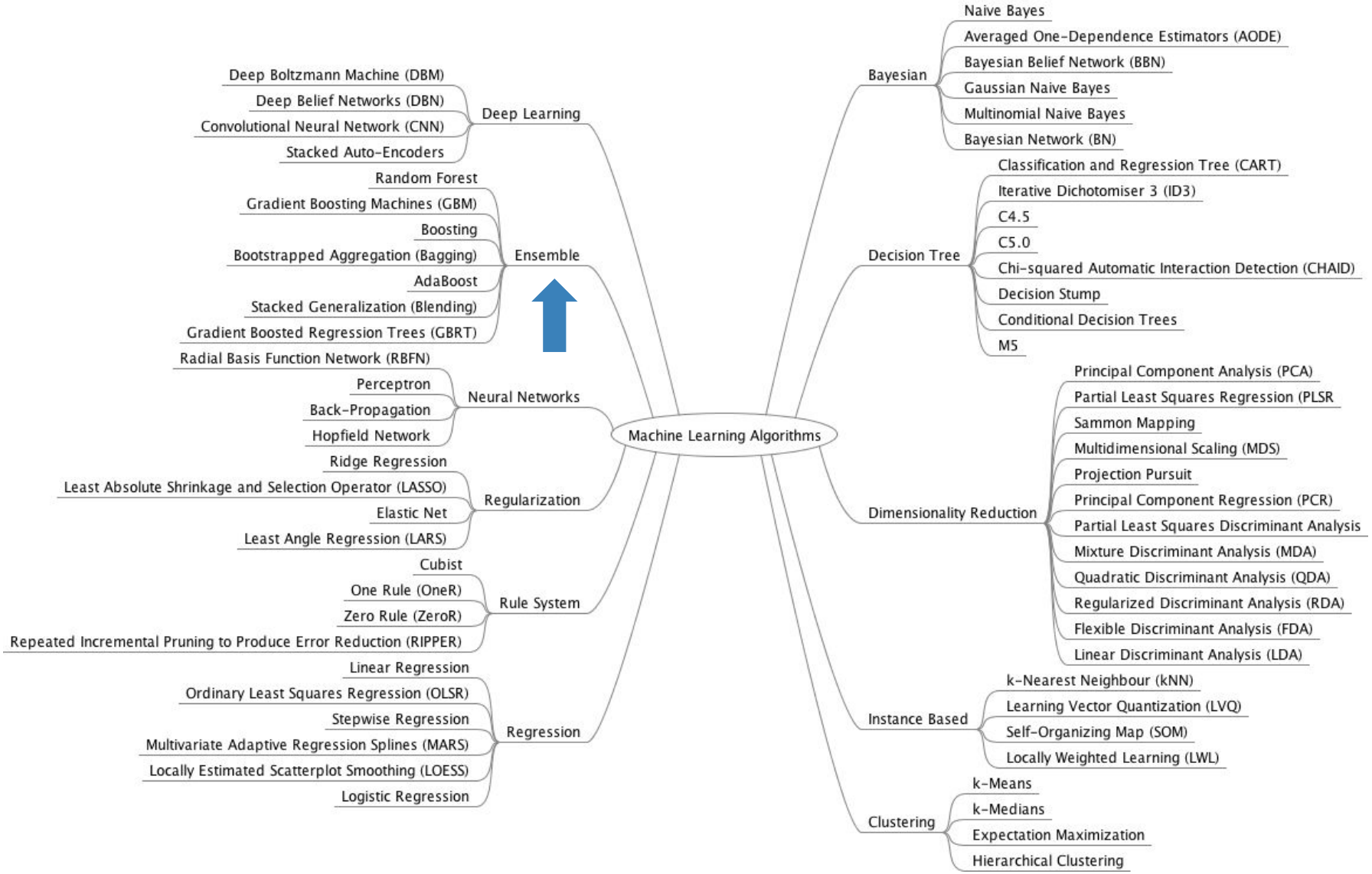
Já sabemos que existem vários tipos de algoritmos de aprendizado preditivo (e.g., árvores de decisão), cada um com suas vantagens e desvantagens

O que aprendemos até agora?

Já sabemos que existem vários tipos de algoritmos de aprendizado preditivo (e.g., árvores de decisão), cada um com suas vantagens e desvantagens

E se combinássemos os indutores em um só?

Métodos Ensemble



1.

Métodos Ensemble

O que são e como funcionam?

Métodos Ensemble

O que é *ensemble*?

Um grupo que produz um único efeito (Dicionário Merriam-Webster)

Jazz ensemble

Corps de ballet

https://www.merriam-webster.com/dictionary/ensemble

Métodos Ensemble

E no contexto de aprendizado de máquina?

É uma classe de algoritmos onde um grupo de modelos individuais (geralmente chamados de modelos base) são **combinados para tomar uma decisão geral**

- “A união faz a força”
- Consultar vários especialistas para tomar uma decisão

Métodos Ensemble

E no contexto de aprendizado de máquina?

É uma classe de algoritmos onde um grupo de modelos individuais (geralmente chamados de modelos base) são **combinados para tomar uma decisão geral**

- "A união faz a força"
- Consultar vários especialistas para tomar uma decisão

É amplamente aceito que **o desempenho de um conjunto de muitos classificadores fracos é geralmente melhor do que um único classificador**, dada a mesma quantidade de informação de treinamento

Métodos Ensemble

Condições para um bom ensemble:

1. **Diversidade:** Os classificadores base devem ser independentes entre si. O ideal é que eles cometam erros em instâncias diferentes dos dados.
2. **Acurácia:** O desempenho de cada classificador individual deve ser, no mínimo, melhor do que um palpite aleatório.

Métodos Ensemble

Um exemplo:

Imagine que temos três classificadores induzidos para os mesmos dados, cada um com 60% de acurácia

- Se eles cometem os mesmos erros, a acurácia do ensemble será igual
- Se forem independentes, o ensemble só errará **se pelo menos dois deles errarem simultaneamente**

Métodos Ensemble

Um exemplo:

Imagine que temos três classificadores induzidos para os mesmos dados, cada um com 60% de acurácia

- Se eles cometem os mesmos erros, a acurácia do ensemble será igual
- Se forem independentes, o ensemble só errará **se pelo menos dois deles errarem simultaneamente**

Grande vantagem do ensemble!

Como combinar predições?

Forma mais simples:

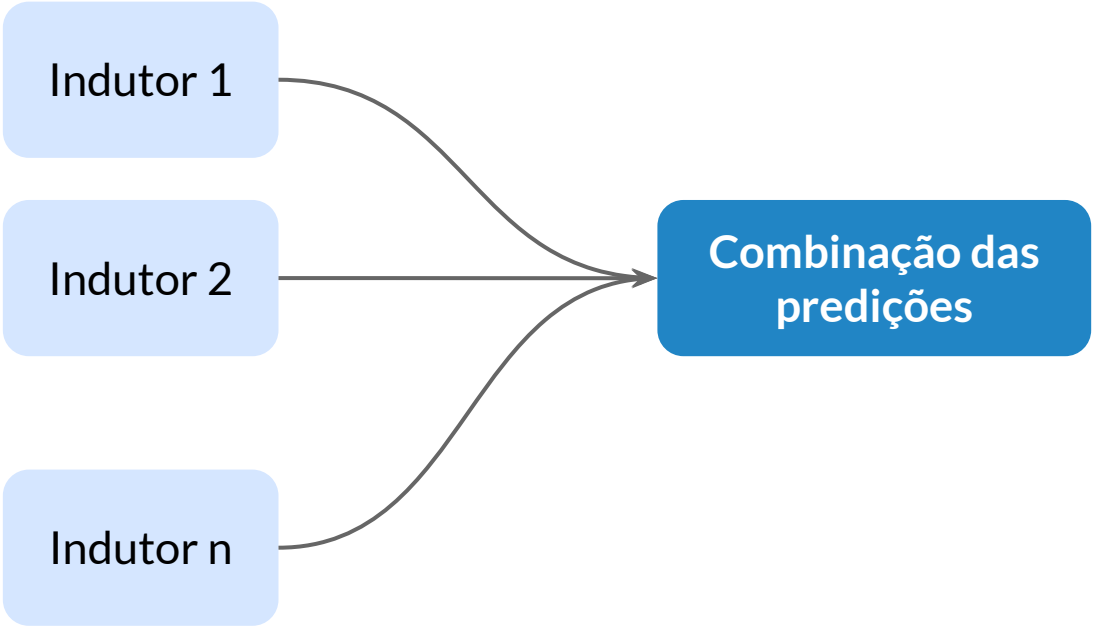
- **Classificação:** por votação (cada modelo "vota" em uma classe e a mais votada vence)
- **Regressão:** por média

Tudo isso pode ser **ponderado**, ou seja, atribui-se pesos diferentes aos modelos com base em sua confiabilidade ou desempenho individual

Como combinar predições?

Forma mais simples:

- **Classificação:** por votação (cada modelo "vota" em uma classe e a mais votada vence)
- **Regressão:** por média



Tipos de Ensemble

Existem alguns tipos de ensemble, mas os mais importantes e utilizados são:

- **Bagging**
- **Boosting**
- **Stacking**

2.

Bagging

O que é e como funciona?

Bagging

Bagging = *bootstrap aggregating*

Foca na **redução da variância** dos modelos base

Bagging

Ideia geral:

1. Criar **múltiplos subconjuntos do conjunto de treino**
   - Bootstrap → tamanho igual ao conjunto original e amostragem com repetição

Bagging

Ideia geral:

1. Criar **múltiplos subconjuntos do conjunto de treino**
   - Bootstrap → tamanho igual ao conjunto original e amostragem com repetição
2. Treinar **um indutor em cada subconjunto**
   - Modelos treinados de forma **independente e simultânea**

Bagging

Ideia geral:

1. Criar **múltiplos subconjuntos do conjunto de treino**
   - Bootstrap → tamanho igual ao conjunto original e amostragem com repetição
2. Treinar **um indutor em cada subconjunto**
   - Modelos treinados de forma **independente e simultânea**
3. Combinar as predições
   - Classificação → votação (classe majoritária)
   - Regressão → média

Bagging

Ideia geral:

1. Criar **múltiplos subconjuntos do conjunto de treino**
   - Bootstrap → tamanho igual ao conjunto original e amostragem com repetição
2. Treinar **um indutor em cada subconjunto**
   - Modelos treinados de forma **independente e simultânea**
3. Combinar as predições
   - Classificação → votação (classe majoritária)
   - Regressão → média

Bootstrap

No método bootstrap, são gerados *r* subconjuntos de treinamento a partir do conjunto original

Os exemplos são amostrados aleatoriamente desse conjunto **com reposição**

- Ou seja, um exemplo pode estar presente em um mesmo subconjunto de treinamento **mais de uma vez**
- É diferente da amostragem aleatória, que geralmente é feita sem reposição

Bootstrap e0

Existem vários métodos *bootstrap*, mas o mais comum é chamado de **e0**

Neste método, cada conjunto de treinamento tem **n exemplos amostrados com reposição** do conjunto original (**n** também é o tamanho do conjunto original)

O que isso significa?

Bootstrap e0

Como amostramos com reposição, nem todos os exemplos E_i podem ir para o conjunto de treinamento, e outros exemplos podem aparecer mais de uma vez

Os exemplos que não aparecerem no treinamento (não foram amostrados) se tornam o **conjunto de teste**

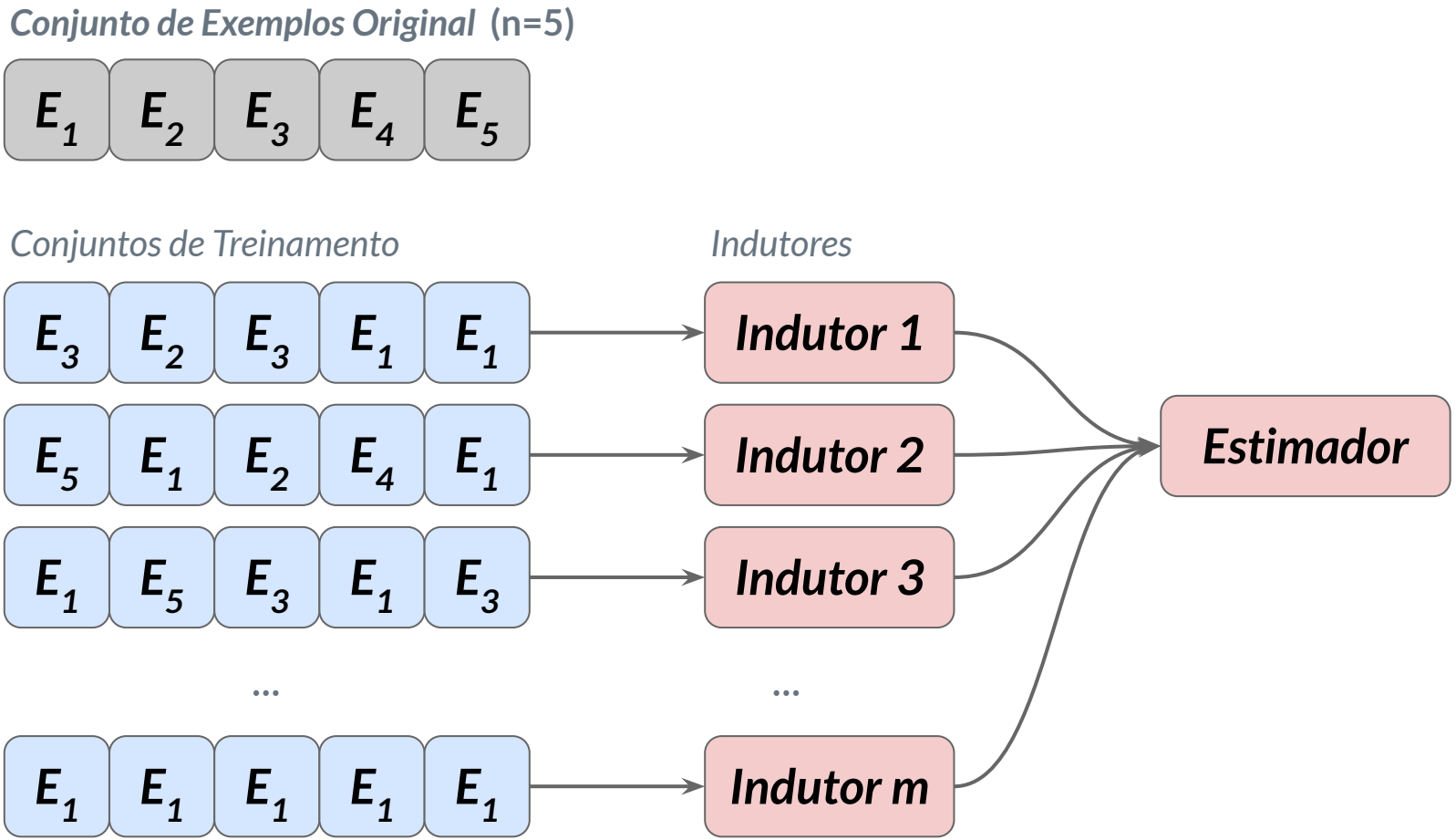
Bootstrap e0

Conjunto de Exemplos Original (n=5)

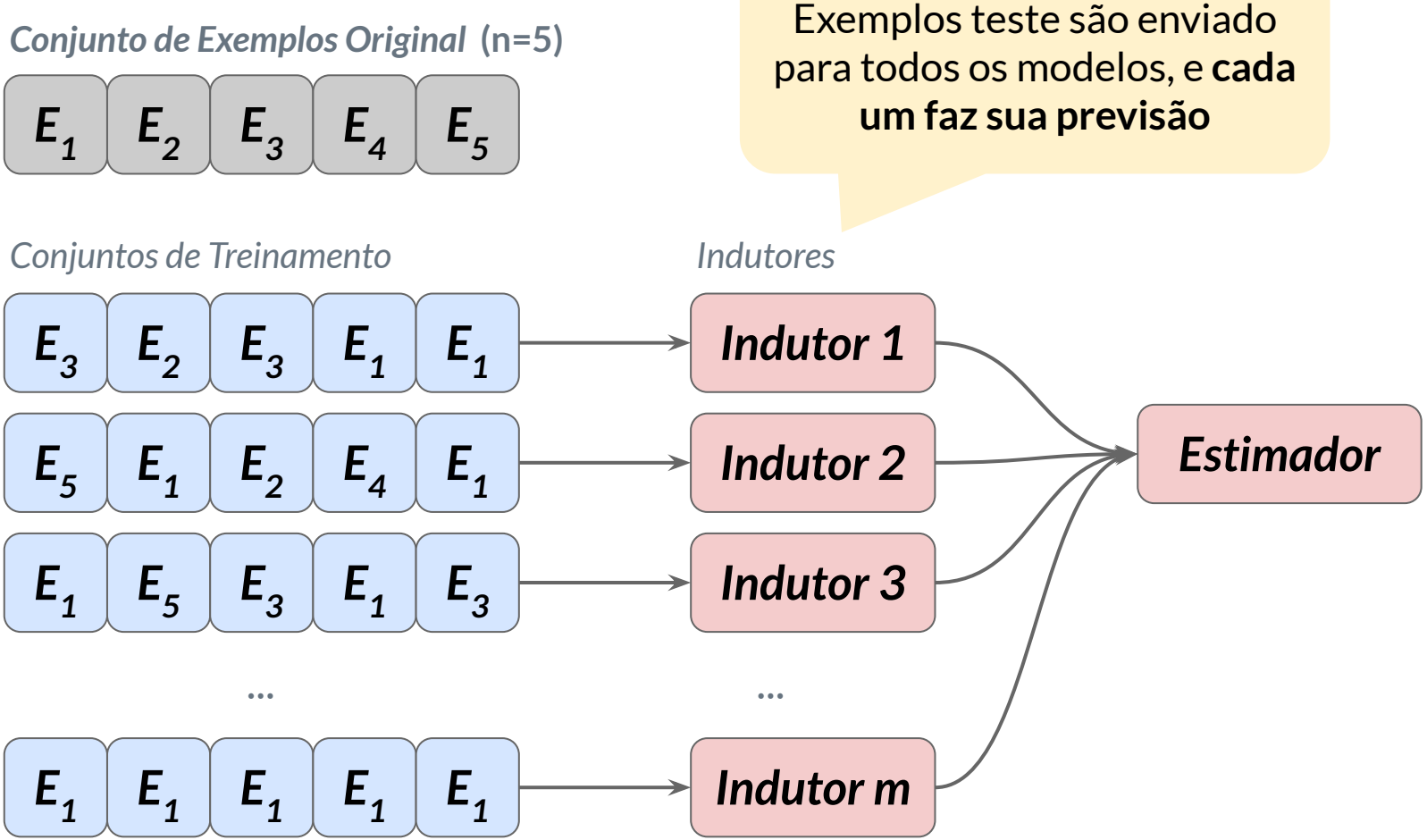
E_1	E_2	E_3	E_4	E_5

Conjuntos de Treinamento					*Conjuntos de Teste*			
E_3	E_2	E_3	E_1	E_1	E_4	E_5		
E_5	E_1	E_2	E_4	E_1	E_3			
E_1	E_5	E_3	E_1	E_3	E_2	E_4		
...					...			
E_1	E_1	E_1	E_1	E_1	E_2	E_3	E_4	E_5

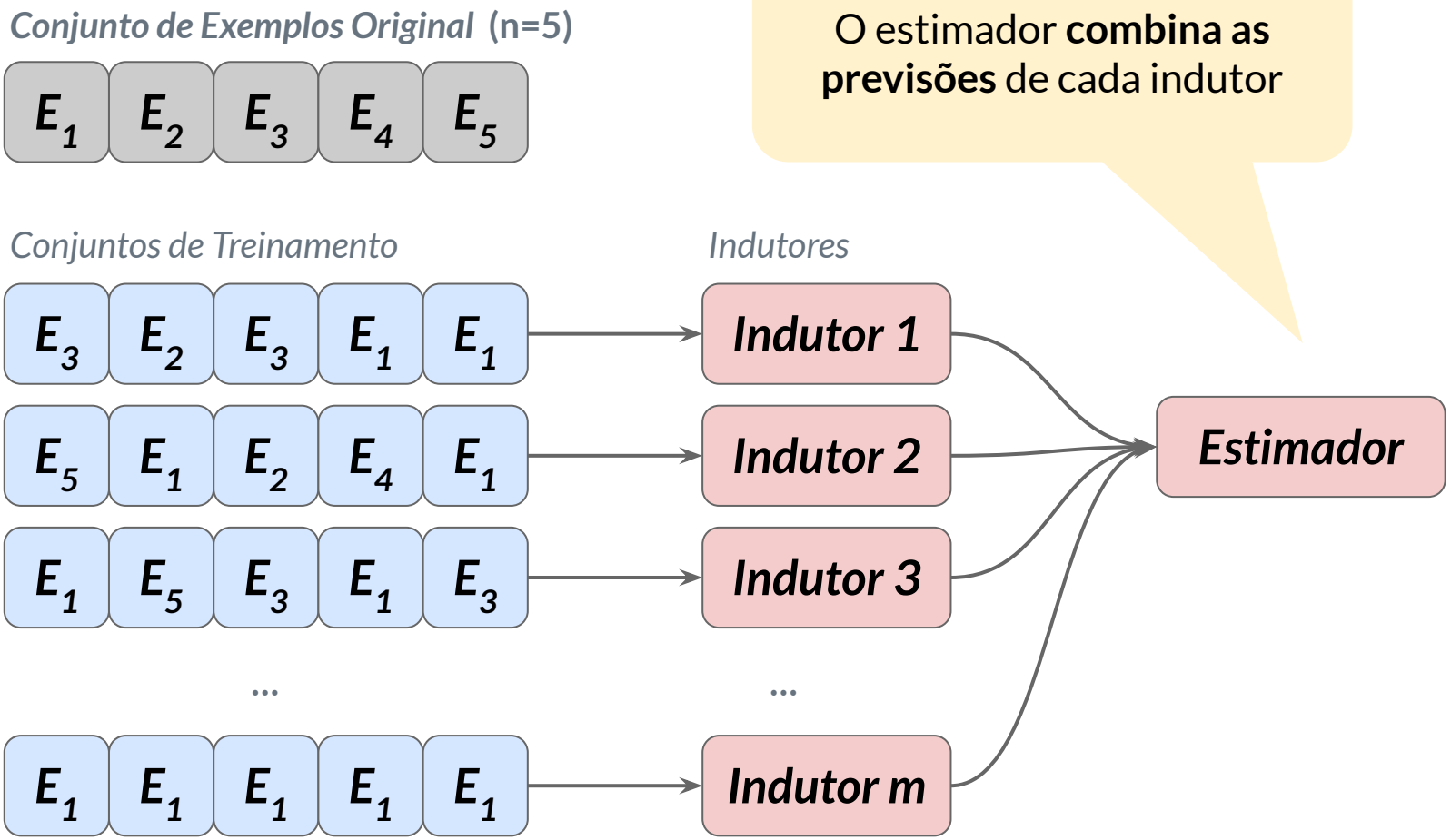
Bagging


Conjunto de Exemplos Original (n=5)
E_1 E_2 E_3 E_4 E_5
Conjuntos de Treinamento
Indutores
E_3 E_2 E_3 E_1 E_1
E_5 E_1 E_2 E_4 E_1
E_1 E_5 E_3 E_1 E_3
...
E_1 E_1 E_1 E_1 E_1
Indutor 1
Indutor 2
Indutor 3
...
Indutor m
Estimador

Bagging


Conjunto de Exemplos Original (n=5)
E_1 E_2 E_3 E_4 E_5
Exemplos teste são enviado para todos os modelos, e **cada um faz sua previsão**
Conjuntos de Treinamento
Indutores
E_3 E_2 E_3 E_1 E_1
E_5 E_1 E_2 E_4 E_1
E_1 E_5 E_3 E_1 E_3
...
E_1 E_1 E_1 E_1 E_1
Indutor 1
Indutor 2
Indutor 3
...
Indutor m
Estimador

Bagging


Conjunto de Exemplos Original (n=5)
E_1 E_2 E_3 E_4 E_5
O estimador **combina as previsões** de cada indutor
Conjuntos de Treinamento
Indutores
E_3 E_2 E_3 E_1 E_1
E_5 E_1 E_2 E_4 E_1
E_1 E_5 E_3 E_1 E_3
...
E_1 E_1 E_1 E_1 E_1
Indutor 1
Indutor 2
Indutor 3
...
Indutor m
Estimador

3.
Boosting

O que é e como funciona?

Boosting

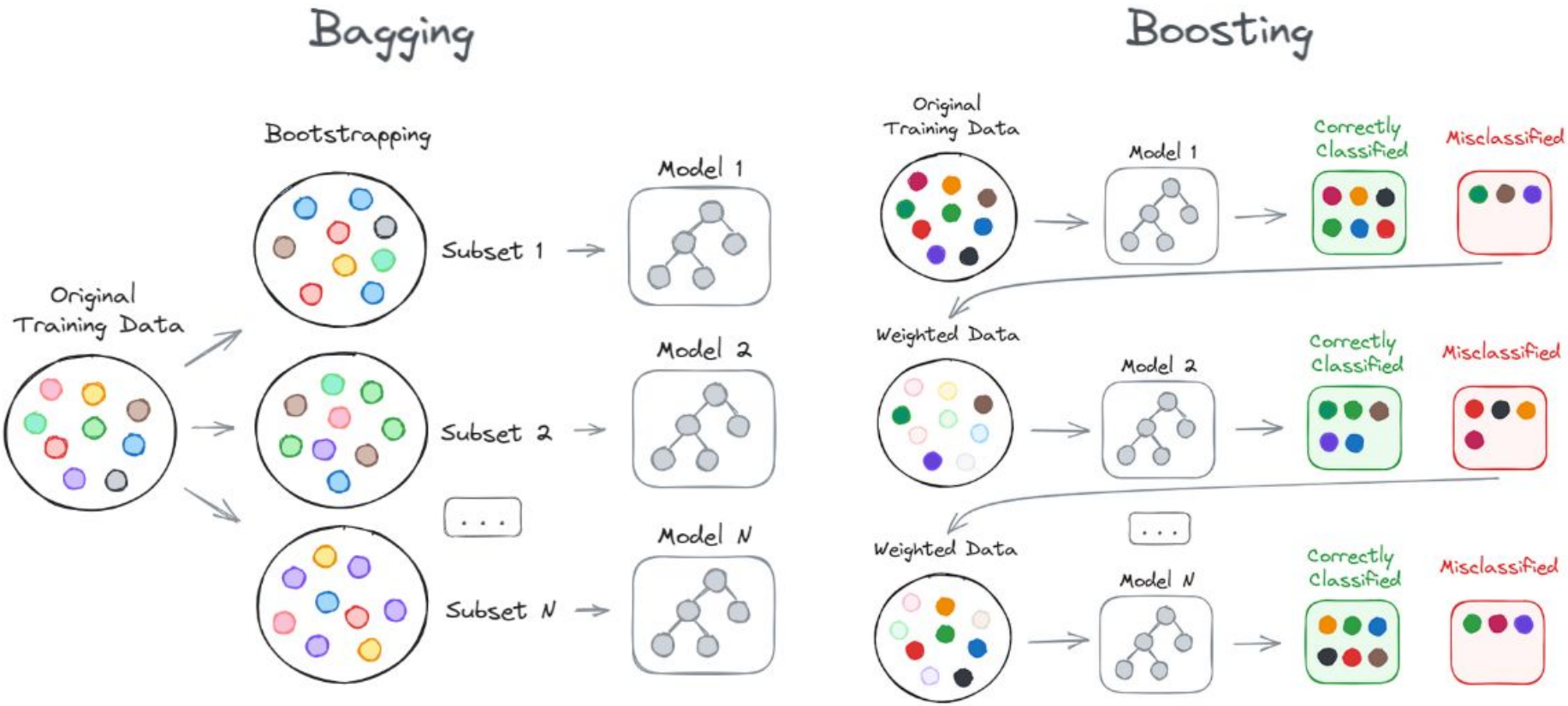
Diferente do bagging, o boosting treina os modelos de **forma sequencial** para reduzir o viés

Boosting

A ideia geral é:

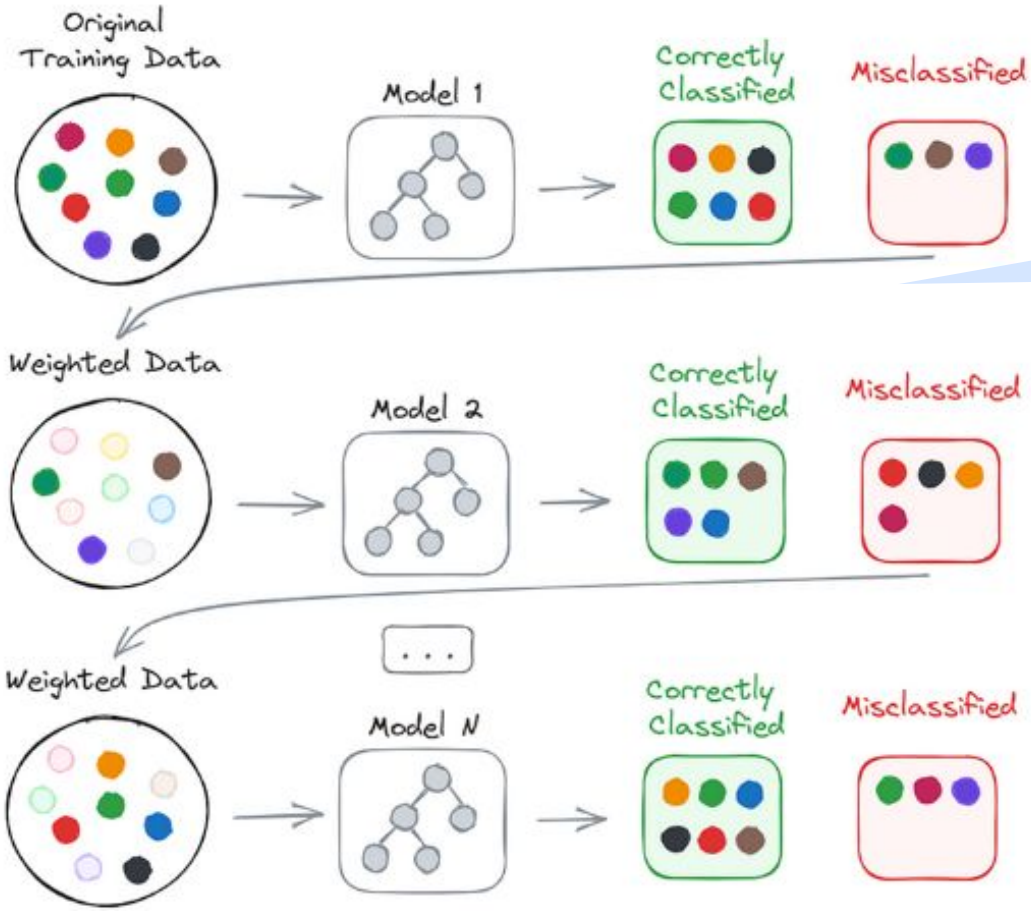
1. Treina um modelo simples
2. Mede onde ele errou
3. Treina um novo modelo **dando mais atenção aos erros do anterior**
4. Repete o processo várias vezes
5. Combina todos os modelos em uma única previsão final

Boosting



Fonte:
https://pub.towardsai.net/bagging-vs-boosting-the-power-of-ensemble-methods-in-machine-learning-6404e33524e6

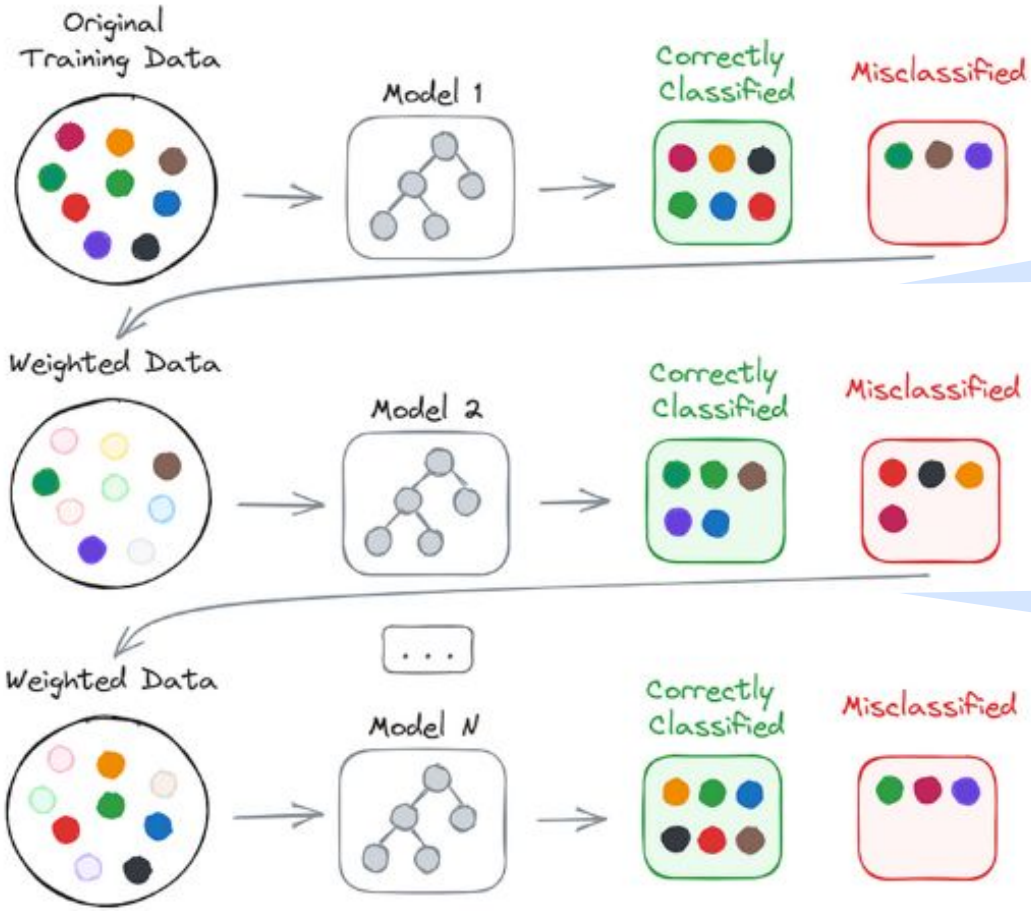
Boosting



Dá um peso maior para os exemplos classificados **incorretamente**

Fonte:
https://pub.towardsai.net/bagging-vs-boosting-the-power-of-ensemble-methods-in-machine-learning-6404e33524e6

Boosting



Dá um peso maior para os exemplos classificados **incorretamente**

Quanto mais difícil de ser aprendido, maior o peso

Fonte:
https://pub.towardsai.net/bagging-vs-boosting-the-power-of-ensemble-methods-in-machine-learning-6404e33524e6

Boosting

Vantagens:

- Indicado para indutores base fracos (acurácia ligeiramente melhor do que o palpite aleatório)
- Convergência rápida

Desvantagens:

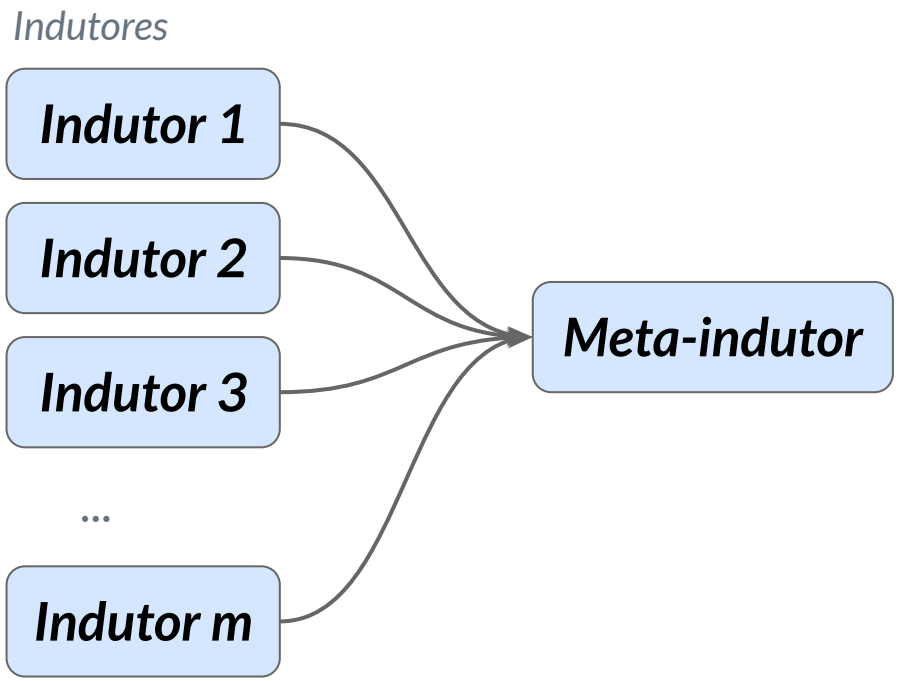
- Não é muito indicado para dados com ruídos e pequenos conjuntos de dados
  - Pelo fato de focar nos exemplos difíceis de serem classificados

4.
Stacking

O que é e como funciona?

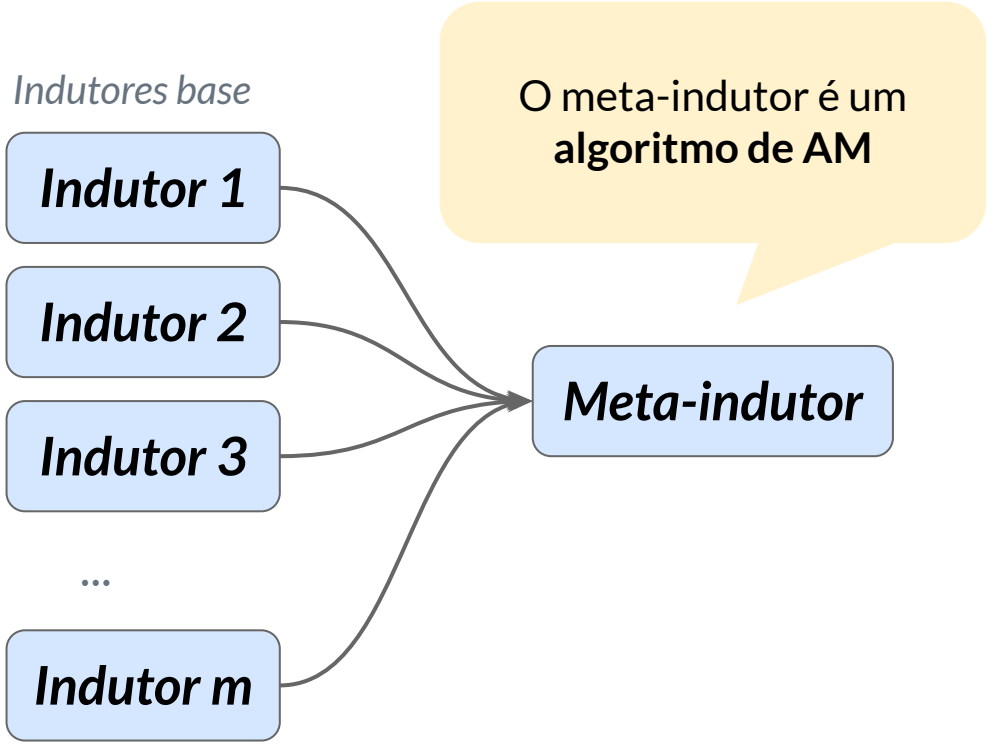
Stacking

Combina vários modelos base diferentes usando um **modelo final** que aprende a melhor forma de combiná-los



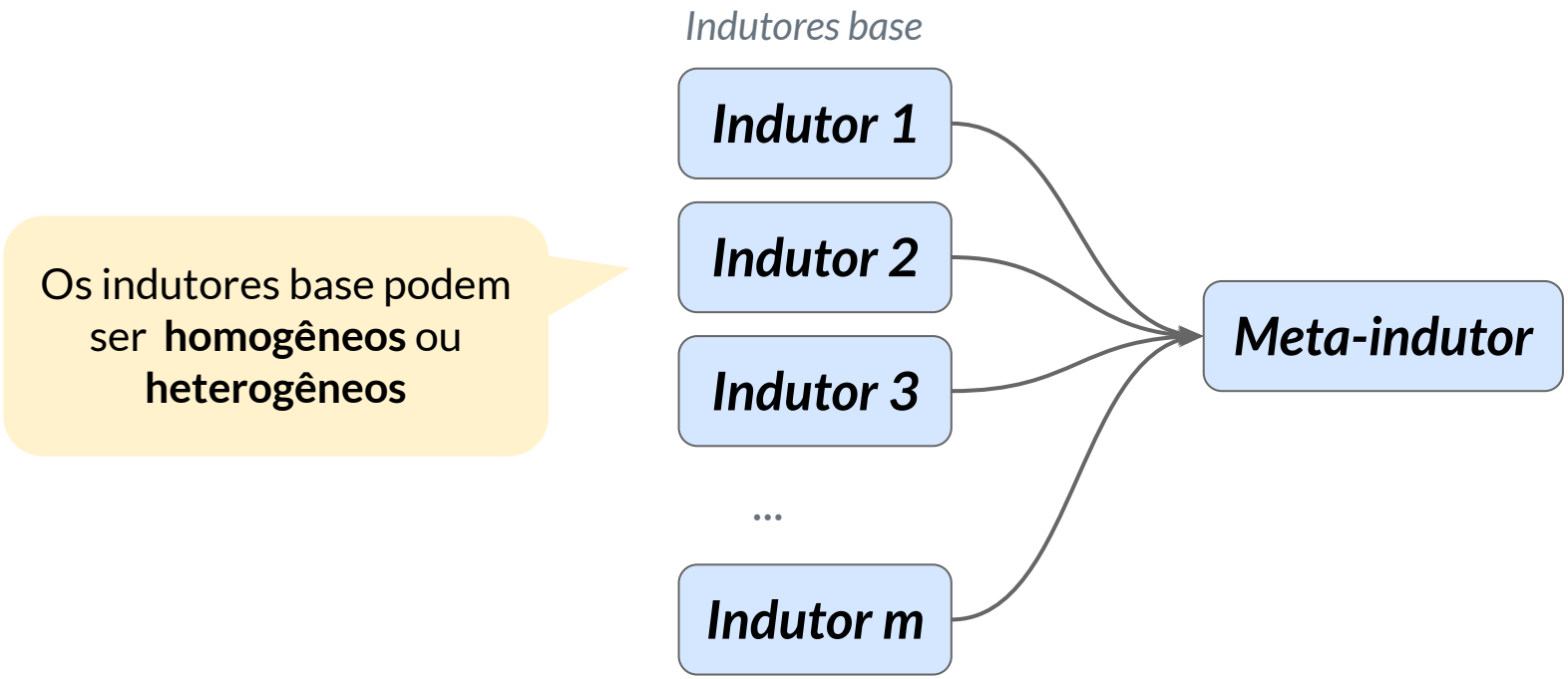
Stacking

Combina vários modelos base diferentes usando um **modelo final** que aprende a melhor forma de combiná-los



Stacking

Combina vários modelos base diferentes usando um **modelo final** que aprende a melhor forma de combiná-los



Em resumo...

	Bagging	Boosting	Stacking
Processo	Agrega múltiplos modelos treinados em diferentes subconjuntos de dados (boostrap)	Constrói modelos sequencialmente e fazem correções de erros baseadas no modelo anterior	Combina predições de múltiplos modelos usando um meta-modelo
Predições	Votação (classificação) e Média (regressão)	Soma ponderada das predições baseadas em cada modelo	Meta-modelo prevê baseado nas previsões dos modelos base
Exemplos	Random Forest	AdaBoost, XGBoost, Gradient Boosting Machines	Regressão Logística em cima de múltiplos classificadores

Na prática...

sklearn.ensemble

Ensemble-based methods for classification, regression and anomaly detection.

User guide. See the Ensembles: Gradient boosting, random forests, bagging, voting, stacking section for further details.

`AdaBoostClassifier`	An AdaBoost classifier.
`AdaBoostRegressor`	An AdaBoost regressor.
`BaggingClassifier`	A Bagging classifier.
`BaggingRegressor`	A Bagging regressor.
`ExtraTreesClassifier`	An extra-trees classifier.
`ExtraTreesRegressor`	An extra-trees regressor.
`GradientBoostingClassifier`	Gradient Boosting for classification.
`GradientBoostingRegressor`	Gradient Boosting for regression.
`HistGradientBoostingClassifier`	Histogram-based Gradient Boosting Classification Tree.
`HistGradientBoostingRegressor`	Histogram-based Gradient Boosting Regression Tree.
`IsolationForest`	Isolation Forest Algorithm.
`RandomForestClassifier`	A random forest classifier.

https://scikit-learn.org/stable/api/sklearn.ensemble.html

4.
Ensembles de Árvores de Decisão

Se uma árvore de decisão é boa... por que não usar várias?

Ensembles de Árvores de Decisão

Dois métodos principais

1. XGBoost (*Extreme Gradient Boosting*)
2. Random Forest

XGBoost

É um algoritmo de **boosting**, ou seja, combina a predição várias árvores de forma **sequencial**

XGBoost

XGBoost: A Scalable Tree Boosting System

Tianqi Chen
University of Washington
tqchen@cs.washington.edu

Carlos Guestrin
University of Washington
guestrin@cs.washington.edu

ABSTRACT

Tree boosting is a highly effective and widely used machine learning method. In this paper, we describe a scalable end-to-end tree boosting system called XGBoost, which is used widely by data scientists to achieve state-of-the-art results on many machine learning challenges. We propose a novel sparsity-aware algorithm for sparse data and weighted quantile sketch for approximate tree learning. More importantly, we provide insights on cache access patterns, data compression and sharding to build a scalable tree boosting system. By combining these insights, XGBoost scales beyond billions of examples using far fewer resources than existing systems.

Keywords

Large-scale Machine Learning

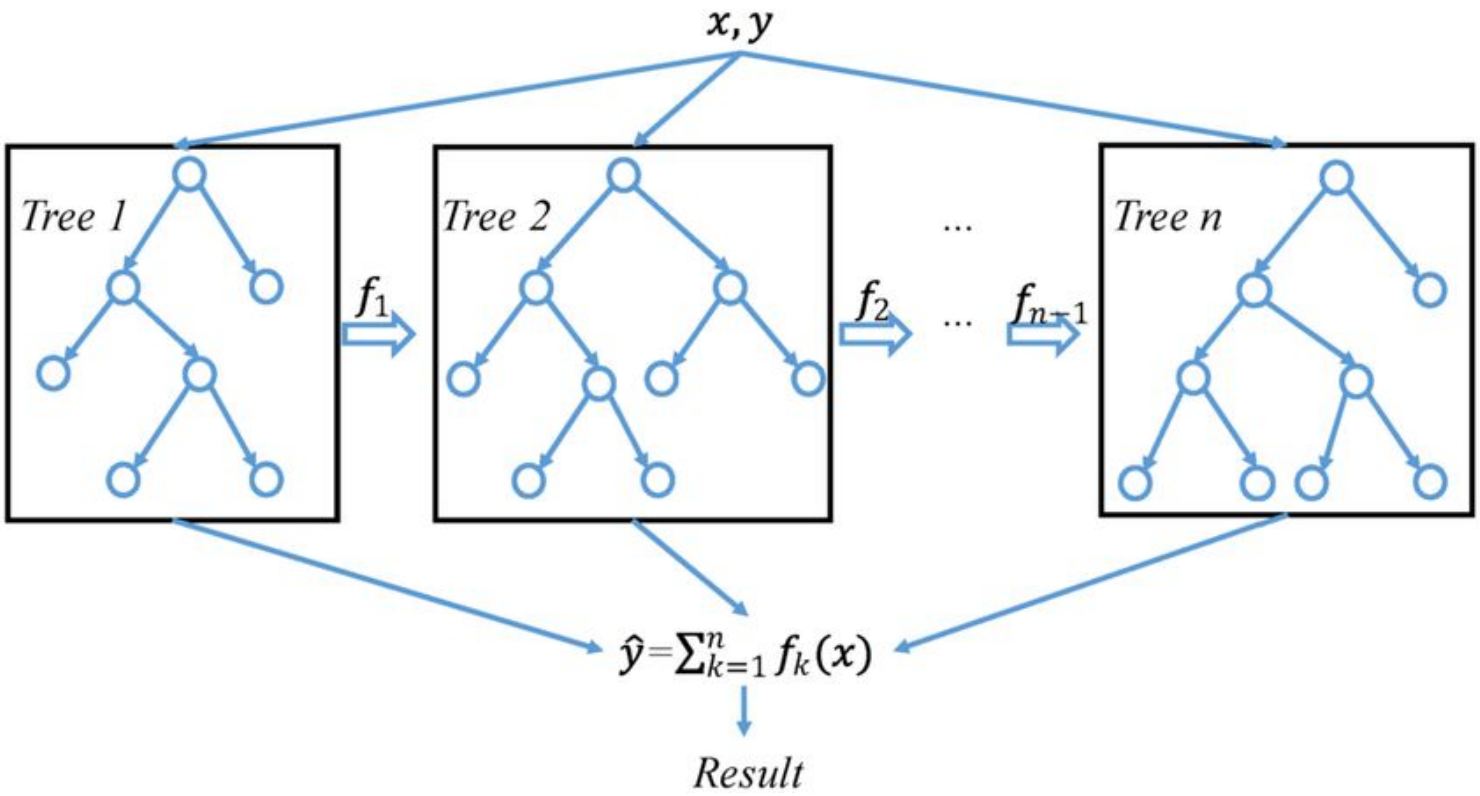
problems. Besides being used as a stand-alone predictor, it is also incorporated into real-world production pipelines for ad click through rate prediction [15]. Finally, it is the de-facto choice of ensemble method and is used in challenges such as the Netflix prize [3].

In this paper, we describe XGBoost, a scalable machine learning system for tree boosting. The system is available as an open source package[2]. The impact of the system has been widely recognized in a number of machine learning and data mining challenges. Take the challenges hosted by the machine learning competition site Kaggle for example. Among the 29 challenge winning solutions [3] published at Kaggle's blog during 2015, 17 solutions used XGBoost. Among these solutions, eight solely used XGBoost to train the model, while most others combined XGBoost with neural nets in ensembles. For comparison, the second most popular method,

https://arxiv.org/pdf/1603.02754

XGBoost

É um algoritmo de **boosting**, ou seja, combina a predição várias árvores de forma **sequencial**

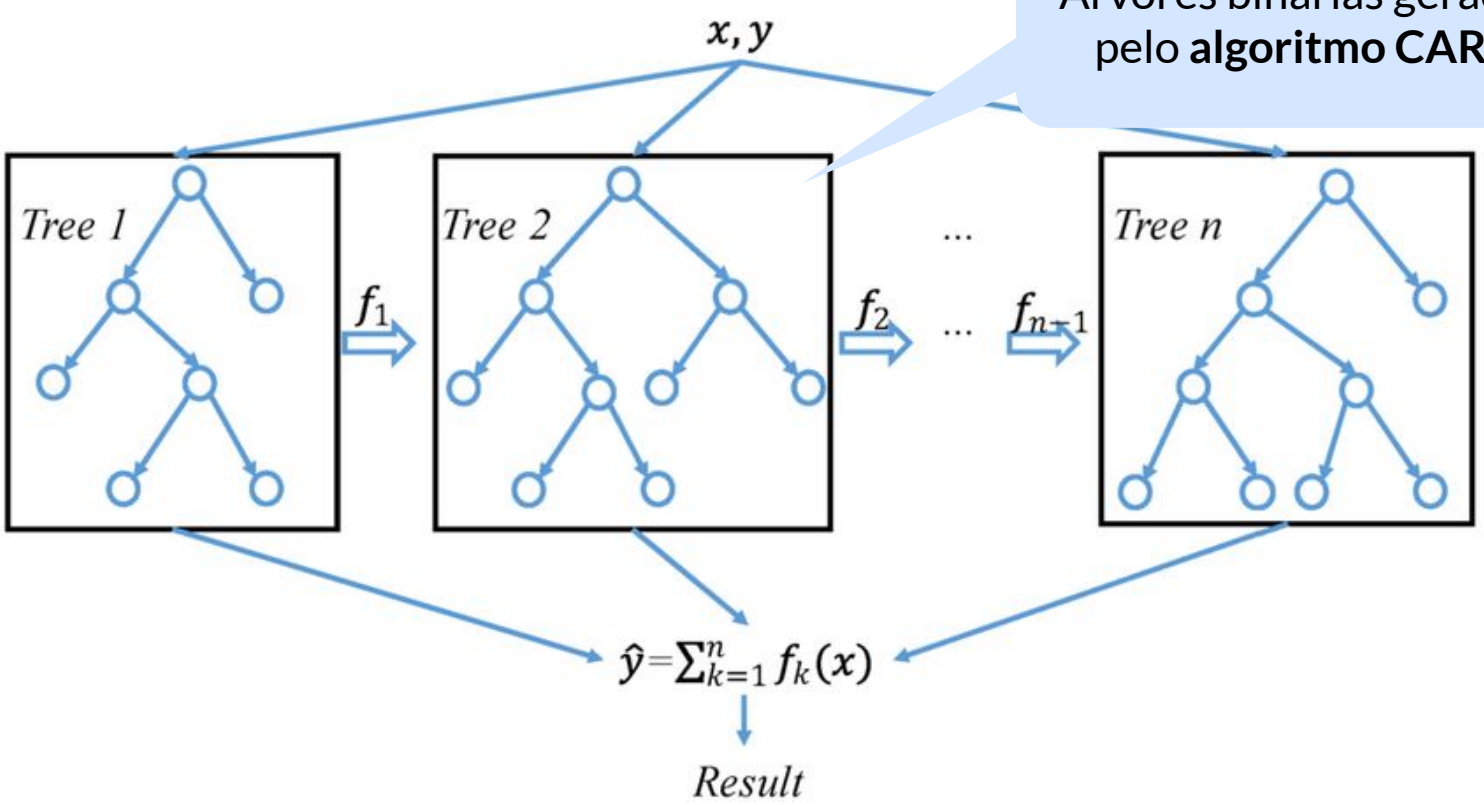


https://doi.org/10.1007/s10509-019-3602-4

XGBoost

É um algoritmo de **boosting**, ou seja, combina a predição várias árvores de forma **sequencial**

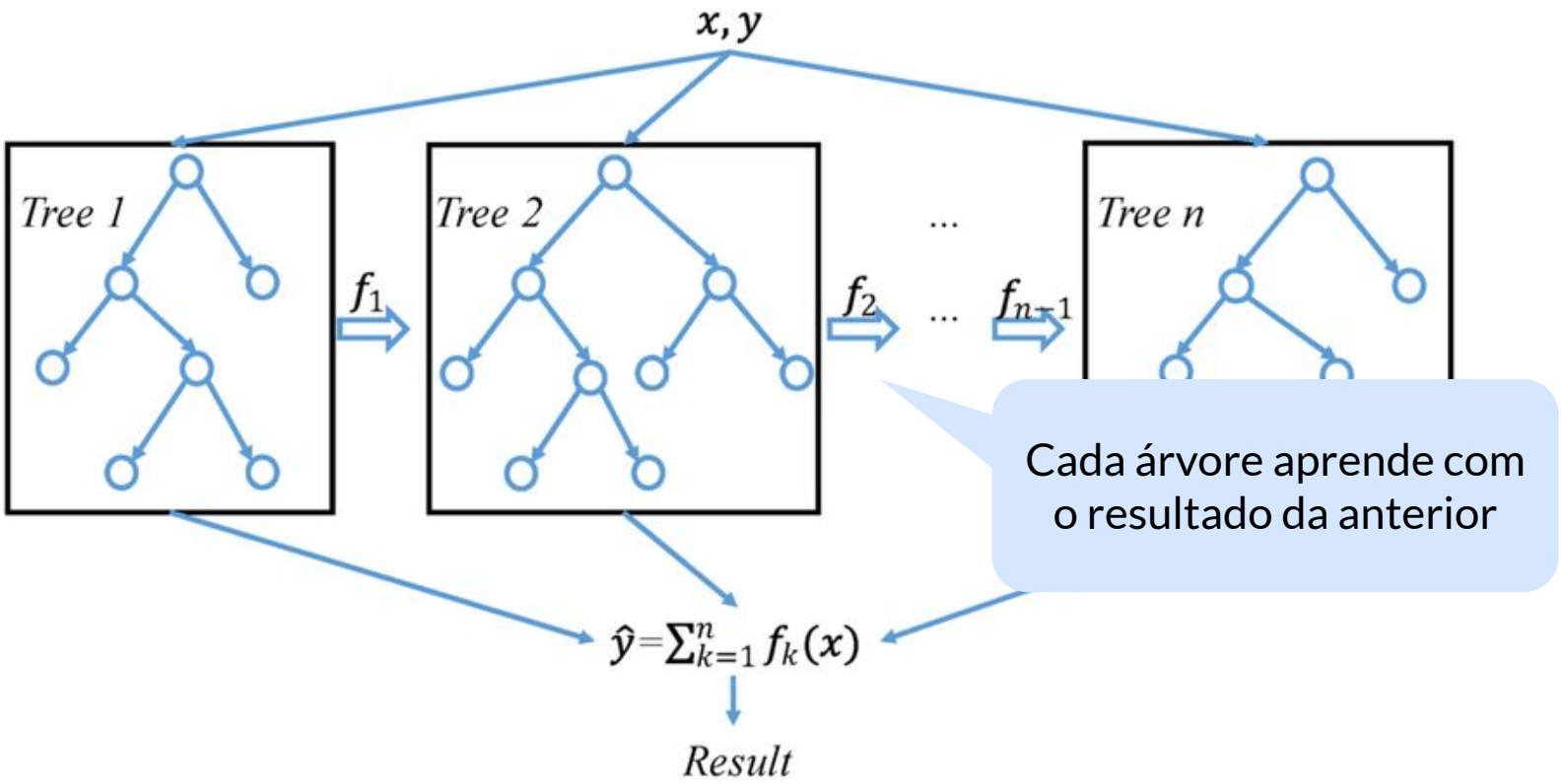
Árvores binárias geradas pelo **algoritmo CART**



https://doi.org/10.1007/s10509-019-3602-4

XGBoost

É um algoritmo de **boosting**, ou seja, combina a predição várias árvores de forma **sequencial**



https://doi.org/10.1007/s10509-019-3602-4

Random Forest

É um algoritmo que utiliza **várias árvores de decisão** utilizadas em paralelo (*bagging*) para prever um valor

Random Forest

É um algoritmo que utiliza **várias árvores de decisão** utilizadas em paralelo (*bagging*) para prever um valor

Machine Learning, 45, 5–32, 2001
© 2001 Kluwer Academic Publishers. Manufactured in The Netherlands.

Random Forests

LEO BREIMAN
Statistics Department, University of California, Berkeley, CA 94720

Editor: Robert E. Schapire

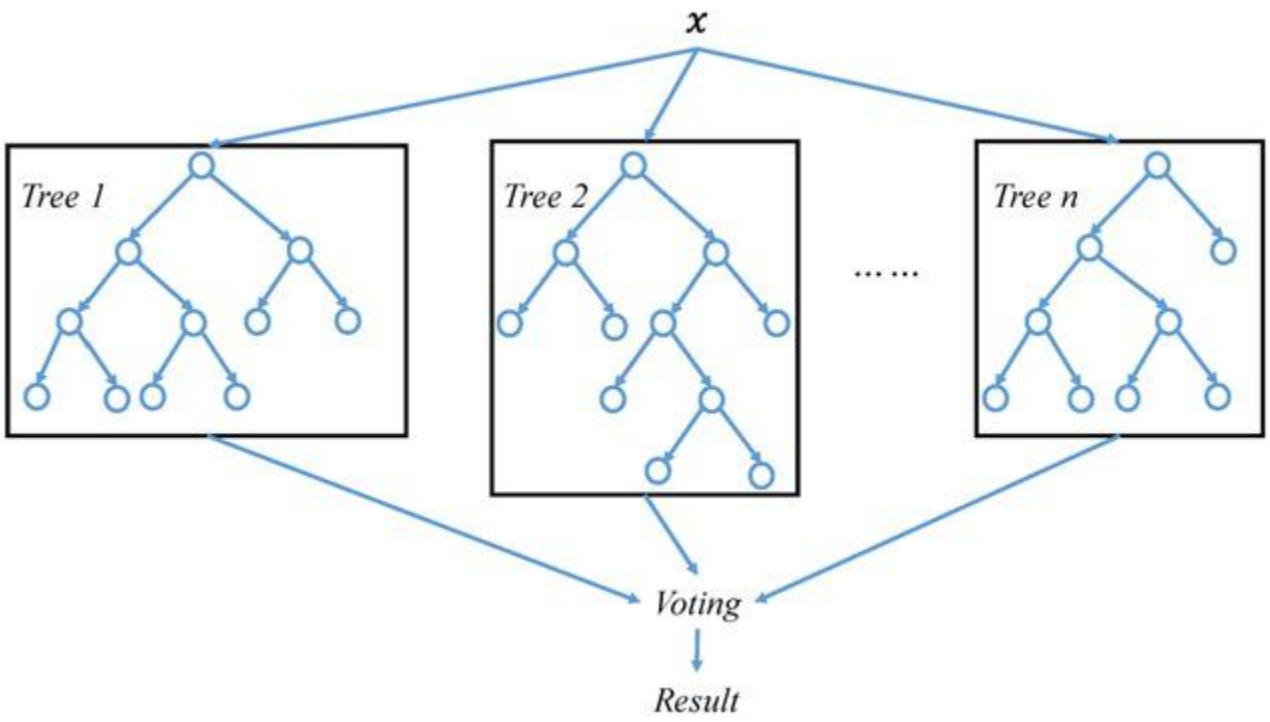
Abstract. Random forests are a combination of tree predictors such that each tree depends on the values of a random vector sampled independently and with the same distribution for all trees in the forest. The generalization error for forests converges a.s. to a limit as the number of trees in the forest becomes large. The generalization error of a forest of tree classifiers depends on the strength of the individual trees in the forest and the correlation between them. Using a random selection of features to split each node yields error rates that compare favorably to Adaboost (Y. Freund & R. Schapire, *Machine Learning: Proceedings of the Thirteenth International conference*, ∗∗∗, 148–156), but are more robust with respect to noise. Internal estimates monitor error, strength, and correlation and these are used to show the response to increasing the number of features used in the splitting. Internal estimates are also used to measure variable importance. These ideas are also applicable to regression.

Keywords: classification, regression, ensemble

https://doi.org/10.1023/A:1010933404324

Random Forest

É um algoritmo que utiliza **várias árvores de decisão** utilizadas em paralelo (*bagging*) para prever um valor



https://doi.org/10.1023/A:1010933404324

Random Forest

Problema do *bagging* tradicional usando árvores: Se houver um atributo muito forte (ex: “renda" para prever crédito), todas as árvores o escolherão para a raiz, tornando-as muito parecidas (correlacionadas)

Random Forest

Problema do *bagging* tradicional usando árvores: Se houver um atributo muito forte (ex: “renda" para prever crédito), todas as árvores o escolherão para a raiz, tornando-as muito parecidas (correlacionadas)

Random Forest possui duas fontes de aleatoriedade:

1. Bootstrap nos dados
2. Subconjunto aleatório de atributos em cada divisão da árvore

Random Forest

Algoritmo para construir uma floresta com T árvores

Para cada árvore:

1. Gerar amostra bootstrap
2. Construir árvore:
    - Em cada nó:
        - Selecionar aleatoriamente *m* atributos
        - Escolher o melhor split apenas entre esses *m*
3. Crescer árvore até critério de parada

Random Forest

Algoritmo para construir uma floresta com T árvores

Para cada árvore:

1. Gerar amostra bootstrap
2. Construir árvore:
   - Em cada nó:
     - Selecionar aleatoriamente *m* atributos
     - Escolher **o melhor split apenas entre esses *m***
3. Crescer árvore até critério de parada

Isso garante que as árvores sejam diferentes!

Random Forest

Qual o valor ideal de *m*?

- **Raiz quadrada** do número de atributos
- **Logaritmo** (base 2) da quantidade de atributos

Random Forest

Random Forest **não faz poda (*pruning*)**, pois o ensemble compensa o crescimento excessivo de cada árvore

Resultado final:

- **Classificação:** classe majoritária
- **Regressão:** média dos valores preditos

Hiperparâmetros importantes

```
from sklearn.ensemble import RandomForestClassifier

modelo = RandomForestClassifier(n_estimators=10, max_features=3,
criterion='entropy', random_state = 0)

modelo.fit(X_treino, y_treino)                # Treina o classificador
modelo.predict(X_teste)          # Faz predição no conjunto de teste
```

Hiperparâmetros importantes

```
from sklearn.ensemble import RandomForestClassifier

modelo = RandomForestClassifier(n_estimators=10, max_features=3,
criterion='entropy', random_state = 0)

modelo.fit(X_treino, y_treino)                    # Treina o classificador
modelo.predict(X_teste)                           # Faz [illegible] te
```

Número de árvores na floresta

Hiperparâmetros importantes

```
from sklearn.ensemble import RandomForestClassifier

modelo = RandomForestClassifier(n_estimators=10, max_features=3,
criterion='entropy', random_state = 0)

modelo.fit(X_treino,   y_treino)                    # Treina o classificador
modelo.predict(X_teste)                # Faz predição para o conjunto de teste
```

Tamanho *m* do **subconjunto aleatório de atributos**

Hiperparâmetros importantes

```
from sklearn.ensemble import RandomForestClassifier

modelo = RandomForestClassifier(n_estimators=10, max_features=3,
criterion='entropy', random_state = 0)

modelo.fit(X_treino, y_treino)                    # Treina o classificador
modelo.predict(X_teste)                  # Faz predição no conjunto de teste
```

Importância dos atributos
'entropy': ganho de informação
'gini': índice de Gini

Hiperparâmetros importantes

```
from sklearn.ensemble import RandomForestClassifier

modelo = RandomForestClassifier(n_estimators=10, max_features=3,
criterion='entropy', random_state = 0)

modelo.fit(X_treino, y_treino)                    # Treina o classificador
modelo.predict(X_teste)                           predição no conjunto de teste
```

Controla aleatoriedade do bootstrap e subconjunto *m*

Vantagens do Random Forest

- Robusto a ruído
- Fornece a importância das variáveis (quais atributos mais contribuem para a decisão)

Desvantagens do Random Forest

- Lento para predições em tempo real se a floresta for gigante
- Comportamento **“caixa preta”**
  - Difícil explicar o porquê de uma decisão específica comparado a uma única árvore

Desvantagens do Random Forest

- Lento para predições em tempo real se a floresta for gigante
- Comportamento **"caixa preta"**
  - Difícil explicar o porquê de uma decisão específica comparado a uma única árvore

Algumas ferramentas úteis para **quantificar o impacto de cada atributo** em uma predição individual:

1. SHAP (SHapley Additive exPlanations)
2. LIME (Local Interpretable Model-agnostic Explanations)

Referências

FACELI, Katti et al. **Inteligência artificial: uma abordagem de aprendizado de máquina.** Rio de Janeiro, RJ: LTC, 2011. (***Capítulo 8)***

Material da Profa. Cristiane Nobre

Breiman, L. Random Forest. Machine Learning (2001) 45: 5. https://doi.org/10.1023/A:1010933404324

Chen, T & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In: KDD. pp. 785–794. https://doi.org/10.1145/2939672.2939785

https://www.stat.berkeley.edu/~breiman/RandomForests/cc_home.htm#workings